Introduction: Functor & Lambda

Parallelism course

October 8, 2026

Class

```
class Vec2i {
    int x; // Private
    int y; // Private
public:
    Vec2i(int x, int y) // Constructor
        : x(x)
        , y(y)
    {}
};
```

Similar to a `struct`, but the default visibility is different (`class`: private, `struct`: public).

Class: Operators

A class can define various operators:

```
class Vec2i {
    // ...
public:
    Vec2i operator*(int value) const {
        return Vec2i{ x * value,
                      y * value };
    }
};

// ...

const Vec2i v1{ 1, 2 };
const Vec2i v2 = v1 * 3; // v2 = Vec2i{ 3, 6 }
```

Functor

Functor: a class defining a call operator `()`:

```
class MyFunctor {
    // ...
public:
    int operator()(int value) const {
        return value + 3;
    }
};

// ...

const MyFunctor functor{};
const int x = functor(5); // Uses the () operator
```

Functor: usage

Standard algorithms use the call operator, which allows passing functors:

```
const std::vector<int> v1{ 2, 4, 6, 8 };
std::vector<int> v2(v1.size(), 0);
const MyFunctor functor{};

std::transform(
    v1.cbegin(), v1.cend(),
    v2.begin(), functor
);
// v2 = { 5, 7, 9, 11 }
```

Functor: signature

Different signature depending on the intended use (two distinct functors):

```
int operator()(int value) const {
    // New value: std::transform
    return value + 3;
}

void operator()(int& value) const {
    // In-place modification: std::for_each
    value += 3;
}
```

Functor: internal state

Advantage: can use a state defined at construction

```
class Adder {
    int value;
public:
    Adder(int value): value(value) {}
    void operator()(int& x) const {
        x += value;
    }
};

// ...

const Adder adder{ 2 };
std::vector<int> v{ 2, 4, 6, 8 };
std::for_each(v.begin(), v.end(), adder);
```

Lambda

Syntax: `[capture](parameters) { body }`

```
std::vector<int> v{ 2, 4, 6, 8 };

// Defined and used in place: no need for a named class
std::for_each(v.begin(), v.end(), [](int& x) {
    x += 3;
});
// v = { 5, 7, 9, 11 }
```

- The return type is deduced from the `return`
- Each lambda has its own anonymous type: use `auto`

Lambda: capture

The [] lists which outer variables can be used:

```
int a = 1, b = 2;

auto f1 = [a](int x) { return x + a; };     // a copied
auto f2 = [&b](int x) { return x + b; };    // b by reference
auto f3 = [=](int x) { return x + a + b; };// everything copied
auto f4 = [&](int x) { return x + a + b; };// everything by reference
```

- Copy: the value is frozen when the lambda is created
- Reference: sees later modifications, but the variable must outlive the lambda

Lambda: `const` and `mutable`

A lambda's `operator()` is `const` by default:

```
int count = 0;

auto f = [count](int x) {
    count += x; // Error: count is const
};
auto g = [count](int x) mutable {
    count += x; // OK, modifies the internal copy
};
auto h = [&count](int x) {
    count += x; // OK, modifies the outer variable
};
```

Lambda: parallelism & pitfalls

Algorithms copy the functor/lambda, and several threads may call it at the same time:

```
std::vector<int> v(1000, 1);
int sum = 0;

std::for_each(std::execution::par, v.begin(), v.end(),
    [&sum](int x) {
        sum += x; // Data race: undefined behavior
    });
```

- Capture by copy: each thread has its own state (modifications are lost)
- Capture by reference: shared state, requires synchronization
- Prefer dedicated algorithms (`std::reduce`, `std::transform_reduce`)

Lambda: equivalence

A lambda is an anonymous functor:

```
int value = 2;
const auto func = [=](int x) {
    return x + value;
};
// ... is equivalent to ...
class __lambda_0 {
    int value;
public:
    __lambda_0(int value): value(value) {}
    int operator()(int x) const {
        return x + value;
    }
};
const __lambda_0 func{ value };
```